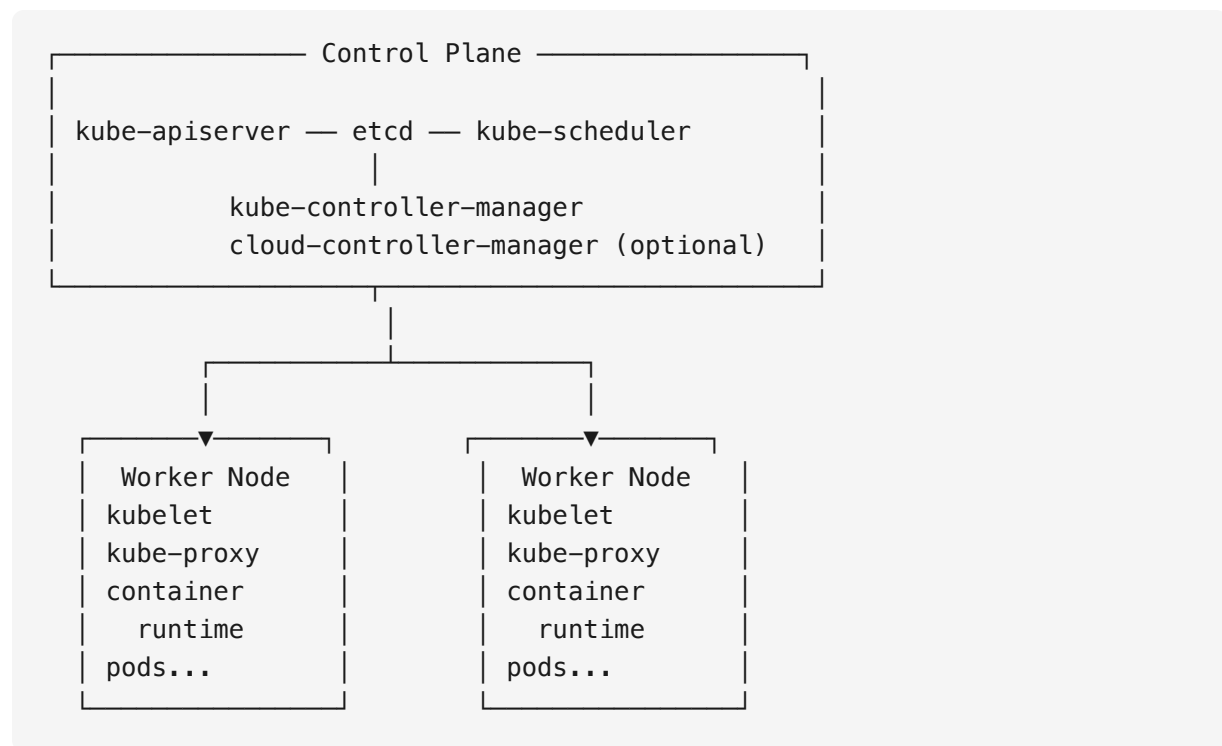


Kubernetes Architecture & Components Cheat Sheet

Adapted from the official [Kubernetes Cluster Components](https://kubernetes.io/docs/concepts/cluster-components/) documentation, © The Kubernetes Authors, licensed under [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). See the official architecture diagram at kubernetes.io/docs/concepts/cluster-components/.

A Kubernetes cluster is made up of a **control plane** and one or more **worker nodes**.



Control Plane Components

Manage the overall state of the cluster — scheduling, responding to events, and reconciling actual state with desired state. Typically run on dedicated control-plane nodes.

Component	Role
kube-apiserver	Front end for the Kubernetes control plane. Exposes the Kubernetes HTTP API; validates and processes all requests.
etcd	Consistent, highly-available key-value store holding all cluster state and configuration data.
kube-scheduler	Watches for newly created Pods with no assigned node and selects a node for them to run on.
kube-controller-manager	Runs the controller processes (node, job, endpoint-slice, service-account controllers, etc.) that reconcile cluster state.
cloud-controller-manager <i>(optional)</i>	Links the cluster to a cloud provider's API, separating cloud-specific logic from core Kubernetes controllers.

Node Components

Run on every node and maintain running pods, providing the Kubernetes runtime environment.

Component	Role
kubelet	Agent that ensures containers described in PodSpecs are running and healthy on the node.
kube-proxy <i>(optional)</i>	Maintains network rules on nodes, implementing part of the Kubernetes Service concept.
Container runtime	Software responsible for running containers (e.g. containerd, CRI-O).

Linux nodes typically also run systemd to supervise these local components.

Addons

Extend cluster functionality using Kubernetes resources (DaemonSets, Deployments, etc.):

- **DNS** (CoreDNS) — cluster-wide service discovery
- **Web UI / Dashboard** — general-purpose cluster management UI

- **Container resource monitoring** — collects and stores container metrics (e.g. metrics-server)
- **Cluster-level logging** — saves container logs to a central log store

Course Cross-Reference

Topic	Where it's covered
kubeadm cluster bring-up, CNI	Day 1
Deployments, ConfigMaps, Secrets, probes	Day 2
Services, Ingress, scheduling, storage	Day 3
RBAC, Pod Security, etcd backup, monitoring	Day 4